ticketing systems, or learning management systems. In these settings, permissions are often determined less by job titles or security groups and more by questions like this: Who owns the resource? Who's part of the project? Who needs to contribute? ReBAC reflects how people typically think about access: through relationships.

At ACME Corp, ReBAC became a critical part of the company's Customer Collaboration platform. This system facilitates complex, dynamic interactions between internal employees—such as solution architects, support engineers, and account managers—and external customers. Access to shared workspaces is determined by account ownership and management chains: employees can grant access to collaborators, and managers automatically inherit access to their reports' accounts. These patterns are difficult to represent clearly with static roles or fragile attribute rules. ReBAC allows ACME to express them using a few simple rules and a shared graph, making access both flexible and easy to understand.

The true power of ReBAC becomes evident as systems grow and evolve. Business teams can handle access themselves using familiar UI actions. Permissions adapt to the structure of the graph, automatically adjusting as relationships change. Governance teams still maintain visibility and control, but without needing to be involved in every decision. ReBAC allows policies to evolve with the organization, making it an ideal foundation for systems centered on collaboration and trust.

6.1.5 When ReBAC falls short

ReBAC systems are powerful, but they aren't always the best choice, especially when access depends on runtime context or complex business conditions. If permissions need to change based on the time of day, device security posture, geographic location, or other transient signals, ReBAC can't evaluate those factors on its own. Trying to embed such logic into the relationship graph can lead to an explosion of relationship types or edges that are hard to validate and understand. Overloading the graph with every condition doesn't just make it more complex—it can also obscure the original intent behind access decisions.